

Labtrack

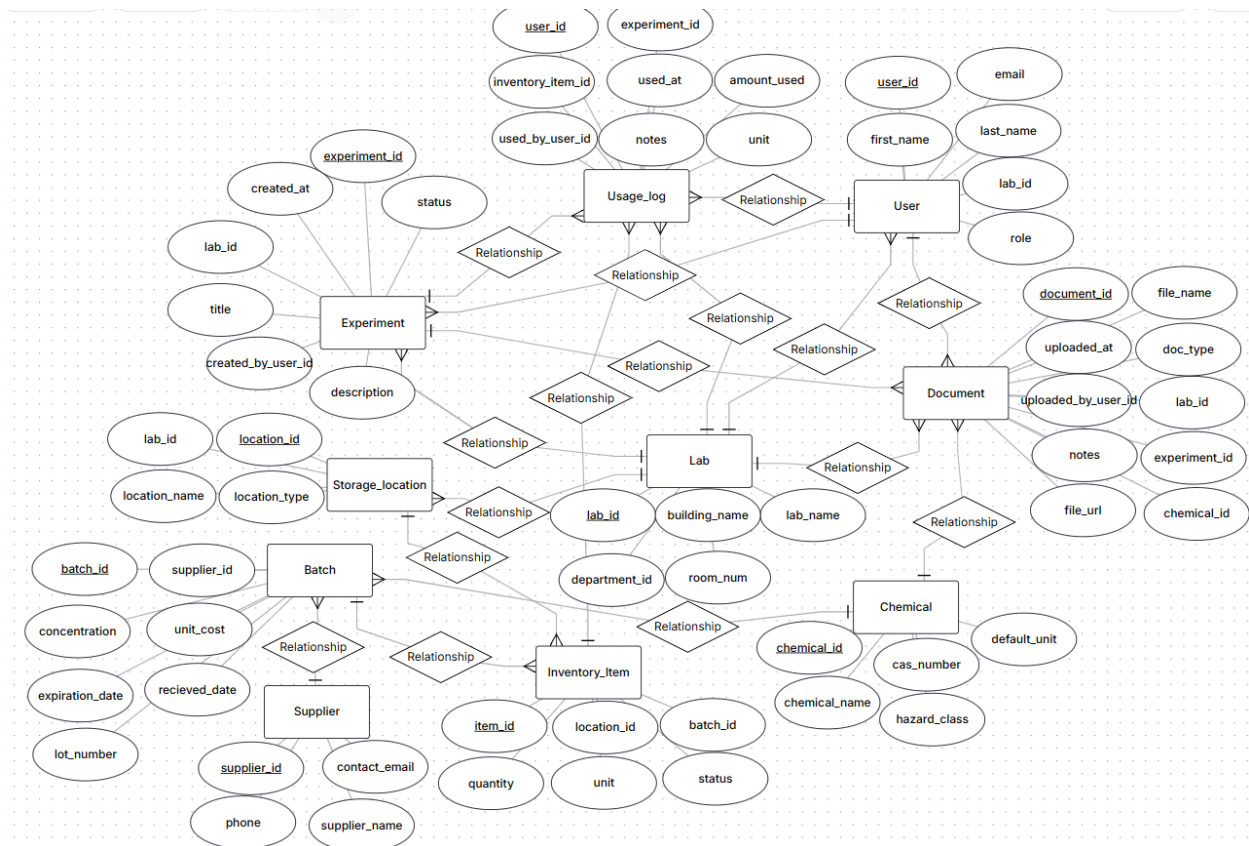
By: Akram Abdelrehem, Junrong Liu, Omar Dovesi, Oscar Aquino Saravia,
Venthai Mao

Application overview

Our application is called Labtrack. It is a chemical inventory management application. The purpose of this application is to give people like lab technicians, or lab managers an easier time to look at the inventory of each item and quickly find out where they are. For lab technicians, they often need to check if certain chemicals are in inventory and where they are, but the process of finding them is very time consuming due to the fact that they might have to check multiple storage locations before finding it. Our application provides a more intuitive interface for them to look up resources much easier.

The targeted audience for our application is lab technicians and lab managers. Many features of our application make it useful. First, it provides an intuitive interface for users to interact with to get the information they need. Secondly, it provides many functionalities such as allowing users to upload new experiment documents, check chemical inventory, and much more, all of which are important for the targeted audiences.

Database design



Major entities and attributes and primary keys:

- **Lab:** lab_id, lab_name, department_id, building_name, room_num
- **User:** user_id, first_name, last_name, email, role, lab_id
- **Chemical:** chemical_id, chemical_name, cas_number, hazard_class, default_unit
- **Supplier:** supplier_id, supplier_name, contact_email, phone
- **Batch:** batch_id, supplier_id, chemical_id, lot_number, received_date, expiration_date, concentration, unit_cost
- **Storage_location:** location_id, lab_id, location_name, location_type
- **Inventory_Item:** item_id, batch_id, location_id, quantity, unit, status
- **Experiment:** experiment_id, lab_id, created_by_user_id, title, description, created_at, status
- **Usage_Log:** usage_id, experiment_id, inventory_item_id, used_by_user_id, used_at, amount_used, unit, notes
- **Document:** document_id, lab_id, uploaded_by_user_id, doc_type, file_name, file_url, uploaded_at, notes, chemical_id, experiment_id

Relationships and constraints:

- **User – (N) – <member_of> – (1) – Lab:** A user can only be a member of one lab and a lab can contain multiple users.
- **Supplier – (1) – <supplies> – (N) – Batch:** A supplier can supply multiple batches but each batch will be associated with only one supplier.
- **Lab – (1) – <contains> – (N) – Storage_location :** Each lab can have multiple storage locations while each storage location needs to be associated with one lab only.
- **Inventory_Item – (N) – <belongs_to> – (1) – Batch:** A batch can contain multiple inventory items while each inventory item is associated with only one batch.
- **Inventory_Item – (N) – <is_located_at> – (1) – Storage_location:** Each inventory item is stored in exactly one storage location; each location can store many inventory items.
- **Experiment – (N) – <conducted_in> – (1) – Lab:** An experiment is associated with at most one lab but each lab can conduct multiple experiments.
- **Experiment – (M) – <conducted_by> – (N) – User:** A user can be associated with multiple experiments (participates in multiple experiments) and each experiment can be associated with multiple users (for collaboration experiment).
- **Experiment – (1) – <create> – (N) – Usage_Log:** An experiment can create many usage log entries, but each usage log entry belongs to exactly one experiment.

- **Usage_Log – (N) – <record_item_used> – (1) – Inventory_Item:** Many usage logs can record one inventory item and one inventory item can appear in many usage logs
- **Usage_Log – (N) – <record_user> – (1) – User:** Many usage logs belong to one user and one user can belong to multiple usage logs.
- **Document – (N) – <upload_by_lab> – (1) – Lab:** A document is associated with one lab but each lab can upload multiple documents.
- **Document – (N) – <uploaded_by_user> – (1) – User:** A document is associated with one user that uploaded the document, and each user can upload multiple documents.
- **Document – (N) – <record_chemical> – (1) – Chemical:** Many documents can record one chemical used in the experiment. And one chemical can belong to multiple documents.
- **Document – (N) – <record_experiment> – (1) – Experiment:** A document can record at most one experiment but each experiment can have multiple documents (because the same experiment can be conducted at different times).
- **Chemical – (1) – <has_batches> – (N) – Batch:** A chemical can have multiple batches, but each batch is for exactly one chemical.

Explanation:

The sections above display the major entities, their primary keys, their relationship with each other, and the constraints of the entity relationships. In terms of how our design supports the functionalities of our application, first, the entity relationships such as Document to Chemical allows our application to be able to associate chemicals used in experiments to the document, allowing users to document their usage on experiment in a document. Document to Lab also allows display of which Lab uploaded the document, helping users to understand which lab used what resources and uploaded the documents, so that if questions arise, they can refer to the lab to ask more questions. Entity relationships such as Experiments to Usage_Log also allows users to easily find out what chemical the experiment used, and allows applications to automatically reduce the amount of chemical inventory in the database, ensuring consistent inventory information, and allows managers to see if a chemical is all used up more quickly.

Database implementation

The LabTrack database is implemented in MySQL 5.7 using a university web server via phpMyAdmin. It consists of 11 tables modeling all core entities of a laboratory management system. The schema enforces referential integrity through foreign-key constraints.

Table	Action	Rows	Type	Collation	Size	Overhead
☐ Batch	★ Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ Chemical	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ Document	★ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	80.0 KiB	-
☐ Experiment	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ Inventory_item	★ Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐ Lab	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ staff_login	★ Browse Structure Search Insert Empty Drop	5	InnoDB	utf8_unicode_ci	16.0 KiB	-
☐ Storage_location	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ Supplier	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ Usage_log	★ Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_general_ci	64.0 KiB	-
☐ User	★ Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_general_ci	48.0 KiB	-
11 tables	Sum	55	InnoDB	utf8_unicode_ci	464.0 KiB	0 B

Table	Purpose
Batch	A specific shipment of one chemical from one supplier, capturing lot number, dates, and cost
Chemical	Master catalog of chemical substances with CAS numbers and hazard classification
Document	Metadata for uploaded files (protocols, SDS sheets) optionally linked to a chemical or experiment
Experiment	Experiment records created inside a lab by a specific user
Inventory_item	A physical container of a batch at a storage location, tracking current quantity and status
Lab	Physical laboratory spaces (building, room, department)

Staff_login	Application authentication table storing hashed passwords, kept separate from User
Storage_location	Named physical storage spots (cabinets, shelves, fridges) within a lab
Supplier	Chemical vendor directory with contact information
Usage_log	Audit trail of chemical usage per experiment — records who used what, how much, and when
User	Staff accounts with assigned roles, each belonging to one lab

Relationship Summary

- **User ↔ Lab (N:1)** — A user belongs to exactly one lab; a lab contains many users.
- **Supplier ↔ Batch (1:N)** — A supplier can provide many batches; each batch comes from one supplier.
- **Chemical ↔ Batch (1:N)** — A chemical can have many batches; each batch is for one chemical.
- **Lab ↔ Storage_location (1:N)** — A lab has many storage locations; each location belongs to one lab.
- **Batch ↔ Inventory_item (1:N)** — A batch can generate many inventory items; each item traces to one batch.
- **Storage_location ↔ Inventory_item (1:N)** — Each inventory item occupies one location; one location can hold many items.
- **Lab ↔ Experiment (1:N)** — A lab hosts many experiments; each experiment is tied to one lab.
- **User ↔ Experiment (M:N)** — An experiment is created by one user; users can collaborate across many experiments. The created_by_user_id column captures the primary creator; broader M:N collaboration can be extended via a junction table.
- **Experiment ↔ Usage_log (1:N)** — Each usage log entry belongs to one experiment; an experiment accumulates many log entries.
- **Inventory_item ↔ Usage_log (1:N)** — Many log entries can reference the same inventory item across experiments.

- **User ↔ Usage_log (1:N)** — Each log entry is recorded by one user; a user may appear in many log entries.
- **Lab ↔ Document (1:N)** — A lab can have many documents; each document is associated with one lab.
- **User ↔ Document (1:N)** — Each document was uploaded by one user; a user may upload many documents.

Constraints and Validation Checks

Constraints	Explanation
UNIQUE on business identifiers	Chemical.cas_number, Supplier.contact_email, and User.email carry UNIQUE constraints, ensuring no two rows represent the same real-world entity.
ON DELETE SET NULL	If a Chemical or Experiment is deleted, the corresponding columns in Document are set to NULL, preserving the file record for audit purposes rather than deleting it.
ON UPDATE CASCADE	All foreign keys cascade primary-key value changes to every referencing row automatically.
NOT NULL on required fields	All mandatory columns (names, dates, amounts, roles) are NOT NULL, preventing incomplete records.
Application-layer validation (PHP)	The PHP backend validates inputs before SQL executes: required fields are checked for emptiness, quantities are verified positive, dates are validated. Passwords are hashed with password_hash() (bcrypt). All queries use PDO prepared statements to prevent SQL injection.

How the Database Was Populated

The database was populated through a SQL script for initial development data, and user-submitted data entered through the LabTrack web application at runtime.

Normalization

A relation is in Third Normal Form (3NF) if and only if it already satisfies the lower two forms:

Normal Form	Requirement
1NF	All column values are atomic (no repeating groups, no multi-valued cells). Each row is uniquely identifiable.
2NF	Every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies). Applies to tables with composite keys.
3NF	No non-key attribute is transitively dependent on the primary key through another non-key attribute. Every non-key attribute depends directly on the key. Nothing but the key. The LabTrack schema satisfies all three forms throughout.

The Chemical table satisfies 3NF because every non-key attribute (chemical_name, cas_number, hazard_class, default_unit, storage_requirement) depends directly and solely on the primary key chemical_id. no non-key attribute determines another. For example, chemical_name does not determine hazard_class in the schema; both depend independently on chemical_id.

T →			chemical_id	chemical_name	cas_number	hazard_class	default_unit	storage_requirement
Edit Copy Delete			1	Ethanol	64-17-5	Flammable	mL	NULL
Edit Copy Delete			2	Acetone	67-64-1	Flammable	mL	NULL
Edit Copy Delete			3	Sodium Chloride	7647-14-5	Irritant	g	NULL
Edit Copy Delete			4	Hydrochloric Acid	7647-01-0	Corrosive	mL	NULL
Edit Copy Delete			5	Sodium Hydroxide	1310-73-2	Corrosive	g	NULL
Edit Copy Delete			6	Glucose	50-99-7	Low Hazard	g	NULL
Edit Copy Delete			7	Potassium	67-67-67	Irritant	g	NULL
Edit Copy Delete			8	Sulfuric Acid	7664-93-9	Corrosive	mL	NULL

Application Functionality & Database Integration

LabTrack is a database-driven web application for managing laboratory chemical inventory, batch tracking, experiments, and documents. Every piece of data such as chemicals, batches, inventory items, usage logs, and experiments is stored in a MySQL relational database. PHP acts as the backend layer, executing SQL queries in response to user actions. The interface reads from and writes to the database in real time: adding a chemical triggers multi-table INSERTs, logging usage updates inventory quantities, and the dashboard reflects live counts from the database on every page.

QUERIES

Example 1 : Insert

```
INSERT INTO Supplier (supplier_id, supplier_name, contact_email)
VALUES (6, 'Northeast Scientific Supply', 'contact@nessupply.com');
DELETE FROM Supplier WHERE supplier_id = 6;
```

Result: The INSERT statement adds a new supplier record into the Supplier table with a specific ID, name, and contact email. The DELETE that follows removes that same row by targeting supplier_id = 6. Together they demonstrate how the application manages supplier data, new suppliers can be added when needed and removed if entered in error, keeping the supplier list accurate for batch tracking.

Example 2 : Inventory Browser

```
SELECT ii.item_id, c.chemical_name, c.hazard_class,
       b.lot_number, b.expiration_date,
       ii.quantity, ii.unit, ii.status, sl.location_name
FROM Inventory_item ii
JOIN Batch      b ON ii.batch_id  = b.batch_id
JOIN Chemical   c ON b.chemical_id = c.chemical_id
JOIN Storage_location sl ON ii.location_id = sl.location_id
WHERE 1=1
ORDER BY b.expiration_date ASC;
```

Result: This SELECT query joins four tables, Inventory_item, Batch, Chemical, and Storage_location to produce a complete inventory view in a single result set. Each row returned shows a chemical's name, hazard class, lot number, expiration date, current quantity, and storage location. The WHERE 1=1 placeholder allows PHP to append optional search or filter conditions dynamically, and ORDER BY b.expiration_date ASC ensures the soonest-expiring items always appear at the top of the inventory table.

Application Interface (GUI)

Layout and Design

LabTrack uses a dark-navy color scheme with a sticky top navigation bar that stays visible on every page. The brand logo and name sit on the left, navigation links run across the center, and the logged-in username with a Sign Out button sit on the right. The dashboard uses a card-based grid layout.

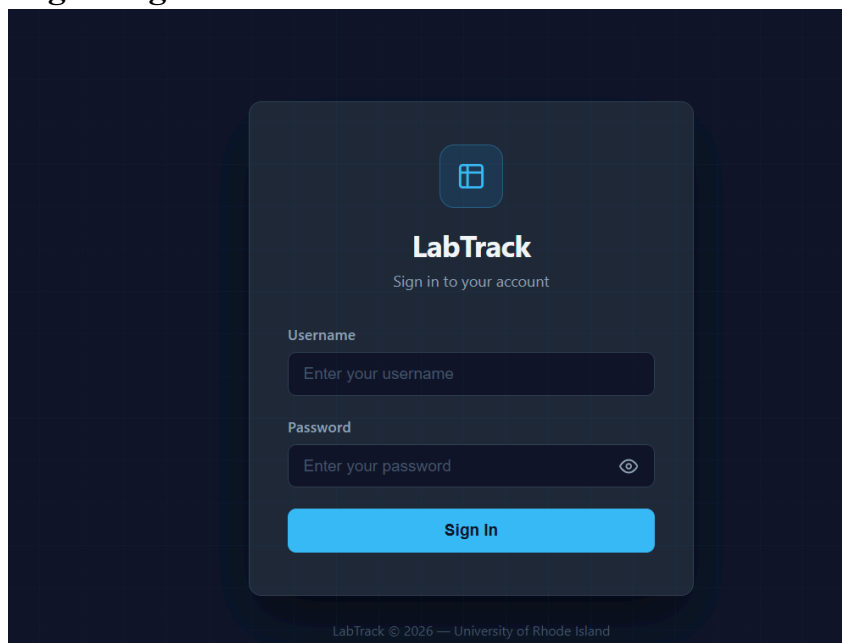
Navigation and Usability

All five main sections such as Dashboard, Inventory, Experiments, Documents, and Chemical Usage are accessible from the top navigation bar with one click. The active page is highlighted with a blue tint so users always know where they are. Each section includes a search bar and filter controls above the data table.

Overall User Experience

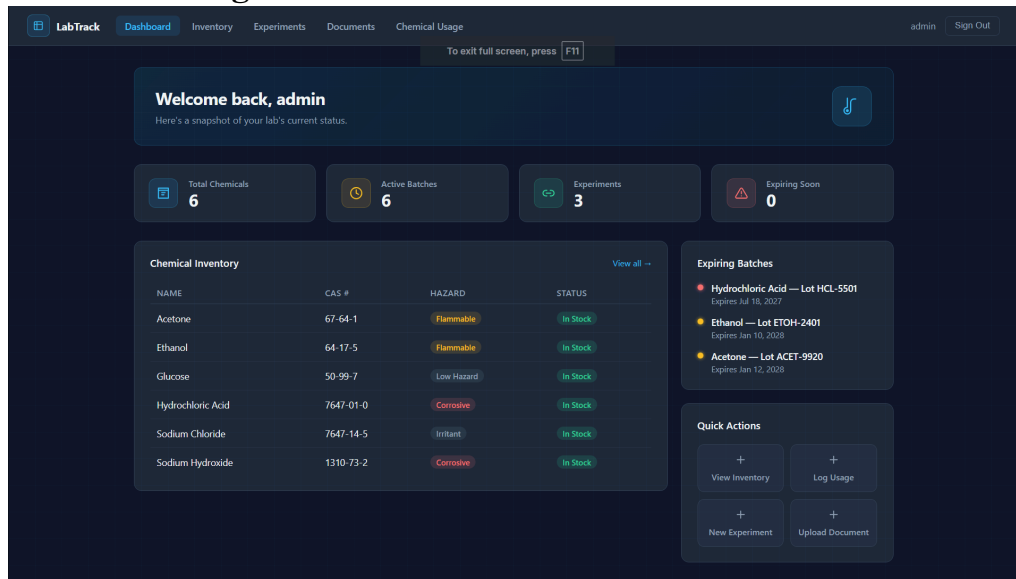
Users go on a login page, authenticate, and are immediately taken to the dashboard which gives a snapshot of the lab's current status. The interface is minimal and task-focused, no clutter, no unnecessary menus. Common actions like logging chemical usage or adding a new batch are reachable in two clicks or fewer.

Login Page:



The screenshot shows the LabTrack login page. It features a dark navy background with a central, slightly lighter navy card. At the top of the card is a blue icon of a laboratory flask. Below the icon, the text "LabTrack" is displayed in a bold, white font, followed by "Sign in to your account" in a smaller, lighter font. The login form consists of two input fields: "Username" and "Password". The "Username" field has a placeholder text "Enter your username". The "Password" field has a placeholder text "Enter your password" and a small eye icon to the right for toggling visibility. Below these fields is a prominent blue "Sign In" button. At the very bottom of the card, in small text, it reads "LabTrack © 2026 — University of Rhode Island".

Dashboard Page:



Inventory Page:

Chemical Inventory

Browse every batch of every chemical — search, filter and log usage in one place.

SEARCH CHEMICAL

e.g. Ethanol

Expiring within 30 days

Search

+ Add Chemical

6 Items shown

#	CHEMICAL NAME	HAZARD CLASS	LOT NUMBER	QUANTITY	LOCATION	EXPIRATION DATE	STATUS	ACTION
1	Hydrochloric Acid	Corrosive	HCL-5501	1,500.00 mL	Corrosives Shelf 1	2027-07-18	In Stock	Log Usage
2	Ethanol	Flammable	ETOH-2401	4,500.00 mL	Flammables Cabinet A	2028-01-10	In Stock	Log Usage
3	Acetone	Flammable	ACET-9920	2,000.00 mL	Flammables Cabinet A	2028-01-12	In Stock	Log Usage
4	Sodium Hydroxide	Corrosive	NAOH-8822	3,000.00 g	Corrosives Shelf 1	2029-01-20	In Stock	Log Usage
5	Glucose	Low Hazard	GLUC-3333	2,500.00 g	Cold Room Fridge	2029-01-25	In Stock	Log Usage

Frontend & Backend Integration

LabTrack is built with HTML and CSS for structure and styling, JavaScript for client-side form validation, PHP as the backend, and MySQL as the database. Every page is server-rendered PHP queries the database on load and injects the results directly into the HTML before it reaches the browser.

Input / Create: Forms (Add Chemical, Add Experiment) submit via HTTP POST to the same PHP page. PHP validates the inputs, then runs parameterized INSERT statements using PDO prepared statements to prevent SQL injection.

Delete / Manage: Suppliers and other records can be removed via DELETE queries targeted by primary key. Documents are uploaded through a form, with the file saved server-side and its metadata INSERTed into the Document table.

Backend: Experiment Database

Frontend: Experiment Page

General Experiments Page

↺

Total Experiments

3

✓

Active

2

🕒

Planned

1

🏆

Completed

0

SEARCH TITLE

STATUS

All statuses

▼

Apply

+ New Experiment

All Experiments				
TITLE	STATUS	LAB	CREATED BY	CREATED
Intro Chem Demo Demonstration for teaching lab	Planned	Chemistry Teaching Lab Room 215	Lina Chen Student Worker	Feb 12, 2026 9:00 am
Solvent Cleaning Protocol Cleaning glassware and equipment	Active	RI-INBRE Core Lab Room 120	Maya Singh Lab Manager	Feb 11, 2026 2:30 pm
Protein Wash Buffer Prep Prepare buffer for imaging workflow	Active	RI-INBRE Core Lab Room 120	Oscar Aquino Admin	Feb 10, 2026 10:00 am

Security, Privacy & Data Protection

In terms of how my group addressed security, privacy and data protection, we did many things. First, when the user tries to log in and the application tries to authenticate if user credentials are correct, we made the application to fetch the user using username only and use prepare statements for it. This helps to prevent SQL injections. Secondly, we hash user passwords when storing passwords in the database and we verify hashed passwords separately from querying for user username. By hashing user passwords when placing passwords into the database, we increase the security of user data and account credentials, so that even if an attacker managed to get into our database, it will be much harder for them to get the password of users. And by using password_verify function to compare user inputted password to the database password, we ensure a more secure comparison and not just comparing plain text passwords, much better than storing passwords as text and comparing them to what users put in.

Optimization & Reliability Enhancements

Identify indexed columns and explain their impact on performance.

Indexed columns include every Primary key like batch_id and chemical_id, and unique columns such as cas_number and email. We also indexed the columns such as lab_id in the User table to allow more efficient JOIN operations.

Identify one important query and explain how it was analyzed, improved, or optimized for efficiency.

Query:

```
SELECT
    ii.item_id,
    c.chemical_name,
    c.hazard_class,
    b.lot_number,
    b.expiration_date,
    ii.quantity,
    ii.unit,
    ii.status,
    sl.location_name
FROM Inventory_item ii
```

```

JOIN Batch      b  ON ii.batch_id = b.batch_id
JOIN Chemical   c  ON b.chemical_id = c.chemical_id
JOIN Storage_location sl ON ii.location_id = sl.location_id
WHERE 1=1;

```

Explanation: It was analyzed by first using the EXPLAIN ANALYZE, to analyze the whole query to test and see its efficiency of the query. And without the indexed columns, the analysis will show that each time, the query will have to scan the whole table for the correct IDs, for example it will need to scan all the rows in the Batch, Chemical, Storage_location table. But with the indexing of the id columns like batch_id, SQL will not be able to look up for the correct ID without scanning the whole table, which is much more efficient.

Identify one multi-step database operation and explain how transactions preserve data integrity and consistency.

Multi-Step Database Operation — Log Chemical Usage

The Log Chemical Usage feature in add_usage.php is a three-step database operation wrapped in a single transaction:

-- Step 1: Lock the row to prevent race conditions

```

// 1. Lock the row and read current quantity (SELECT ... FOR UPDATE)
$lockStmt = $pdo->prepare("
    SELECT quantity
    FROM   Inventory_item
    WHERE  item_id = ?
    FOR UPDATE
");

```

-- Step 2: Record the usage event

```

// 2. INSERT into Usage_log
$insertStmt = $pdo->prepare("
    INSERT INTO Usage_log
    (experiment_id, inventory_item_id, used_by_user_id, used_at, amount_used, unit, notes)
    VALUES
    (?, ?, ?, NOW(), ?, ?, ?)
");

```

-- Step 3: Decrement quantity and update status

```
$updateStmt = $pdo->prepare("
    UPDATE Inventory_item
    SET    quantity = ?,
           status   = ?
    WHERE item_id  = ?
```

How the transaction preserves integrity:

`$pdo->beginTransaction()` starts the transaction. All three steps must succeed before `$pdo->commit()` finalizes any changes. If anything fails — such as a constraint violation, connection drop, or a request exceeding the available stock — `$pdo->rollBack()` undoes every change, leaving the database exactly as it was before the transaction began.

The `SELECT ... FOR UPDATE` statement in Step 1 is especially important because it locks the `Inventory_item` row for the duration of the transaction. This prevents two users from simultaneously logging usage against the same item and both reading the same quantity before either `UPDATE` executes, avoiding a classic race condition.

As a result, `Usage_log` and `Inventory_item.quantity` always remain synchronized. The system can never create a usage record without the corresponding stock reduction, nor reduce stock without creating an audit trail.

Figures

indexed columns:

		chemical_id	chemical_name	cas_number	hazard_class	default_unit
☐	 Edit  Copy  Delete	1	Ethanol	64-17-5	Flammable	mL
☐	 Edit  Copy  Delete	2	Acetone	67-64-1	Flammable	mL
☐	 Edit  Copy  Delete	3	Sodium Chloride	7647-14-5	Irritant	g
☐	 Edit  Copy  Delete	4	Hydrochloric Acid	7647-01-0	Corrosive	mL
☐	 Edit  Copy  Delete	5	Sodium Hydroxide	1310-73-2	Corrosive	g
☐	 Edit  Copy  Delete	6	Glucose	50-99-7	Low Hazard	g

batch_id	supplier_id	chemical_id	lot_number	received_date	expiration_date	concentration	unit_cost
1	1	1	ETOH-2401	2026-01-10	2028-01-10	NULL	45.50
2	2	2	ACET-9920	2026-01-12	2028-01-12	NULL	39.99
3	3	3	NACL-1102	2026-01-15	2030-01-15	NULL	12.75
4	1	4	HCL-5501	2026-01-18	2027-07-18	1.00	28.25
5	2	5	NAOH-8822	2026-01-20	2029-01-20	NULL	18.40
6	3	6	GLUC-3333	2026-01-25	2029-01-25	NULL	15.00

location_id	lab_id	location_name	location_type
1	1	Flammables Cabinet A	Cabinet
2	1	Corrosives Shelf 1	Shelf
3	1	Cold Room Fridge	Fridge
4	2	Teaching Lab Cabinet 3	Cabinet

The query was already pretty optimized since many of the columns used are indexed.

```

1 SELECT
2     ii.item_id,
3     c.chemical_name,
4     c.hazard_class,
5     b.lot_number,
6     b.expiration_date,
7     ii.quantity,
8     ii.unit,
9     ii.status,
10    sl.location_name
11 FROM Inventory_item ii
12 JOIN Batch          b  ON ii.batch_id = b.batch_id
13 JOIN Chemical       c  ON b.chemical_id = c.chemical_id
14 JOIN Storage_location sl ON ii.location_id = sl.location_id
15 WHERE 1=1;
16

```


multi-step database operation:

```
// 1. Lock the row and read current quantity (SELECT ... FOR UPDATE)
$lockStmt = $pdo->prepare("
    SELECT quantity
    FROM   Inventory_item
    WHERE  item_id = ?
    FOR UPDATE
");
```

```
// 2. INSERT into Usage_log
$insertStmt = $pdo->prepare("
    INSERT INTO Usage_log
    (experiment_id, inventory_item_id, used_by_user_id, used_at, amount_used, unit, notes)
    VALUES
    (?, ?, ?, NOW(), ?, ?, ?)
```

```
$updateStmt = $pdo->prepare("
    UPDATE Inventory_item
    SET     quantity = ?,
           status    = ?
    WHERE  item_id   = ?
```

Reflection & Final Improvements

Strengths

The application's strongest area was its clean, consistent database design. Normalizing data across tables like Chemical, Batch, Inventory_item, and Usage_log eliminated redundancy and made queries straightforward. The multi-table JOIN for the inventory browser worked well and returned exactly what the interface needed in a single query.

Future Improvements

Given more time, the team would add role-based access control so that lab managers have edit/delete permissions while standard users can only log usage and view data. A real-time low-stock alert system (email or dashboard notification when quantity drops below a threshold) would make the app more proactive.

Team Contributions

Akram - 10%

Junrong - 20%

Omar - 20%
Oscar - 30%
Venthai - 20%

Part II: Database Application

Application Website: <https://oscaraquino.rhody.dev/labtrack/login.php>